TP3 implementation and course references

This document describes the current implementation. The original decisions and completion plan remain in ‘tp3_decisions.md‘ and ‘tp3_completion_plan.md‘ as design history. The confirmed rules are implemented; the previous pending-clarification messages have been removed. Neither member belongs to the 2011 study plan, so Simple Reduction is not part of this group’s requirements.

Requirements and implementation

Course documents are available in ‘/home/rodrigo/facultad/compi‘.

Source	Requirement and implementation
TP3 pp.1-3; 08.4 slide 11	Dynamic RPN with stable symbol references, types, uses, lexical scopes and declaration-order lookup. Existing scope guards also restore discarded definitions.
TP3 p.5	Independent executable function and method bodies, ordered formal parameter metadata, copy semantics, primitive RET and default zero of the declared primitive type.
TP2 p.4; TP3 p.7	Mandatory evaluation order, matching counts and argument-to-formal association. Argument fragments are copied in the requested order using separate pending frames.
TP2 p.7; TP3 pp.10-11	Positional member access, existence, visibility and inherited lookup. Effective attributes are used consistently for access, copies and structural comparison.
TP3 p.9	Class codes are retained; equal-code classes are checked when complete. Copies between distinct classes require matching codes.
TP3 p.8; 08.3 slides 31-33	Compile-time values and semantic attributes. A post-parse pass evaluates known state and replaces compile-time reads with constants.
TP3 p.6	FROM retains strict comparisons and updates the control variable’s current value after the body. Compile-time-dependent loops process those updates on each expanded iteration.
TP3 p.4; 08.3 slide 66	General runtime numeric compatibility remains in the later RPN-to-assembler translation. Evaluating a compile-time binding requires a representable value of its declared type.
08.3 slide 78	Explicit TOI remains in RPN; the compile-time evaluator can evaluate it when its operand is known.
TP3 p.12	Successful output contains numbered, readable RPN and TS metadata; diagnostics include source lines. Debug traces are opt-in.

The following specific rules were confirmed by the user to complete details not stated explicitly by the course: rejecting compile-time modifications under unknown control; structural equality by attribute type and order; inherited attributes before own attributes; declaration identity in diamonds; each argument position appearing once; and excluding object returns. They are not presented as literal statements from the slides.

Class schemas

Own declarations remain separate from effective attributes. Bases are traversed in source order, recursively placing their inherited attributes first; own attributes follow. This traversal order is an explicit representation convention. It never grants first-base priority when distinct inherited declarations conflict.

The same declaration reached through multiple paths is included once. Distinct homonymous inherited declarations remain subject to topic 32 ambiguity checks. Cycles produce diagnostics instead of leaving a traversal running indefinitely. Member lookup also works from within derived methods, following lexical lookup and the inherited member rules.

Classes with the same code must have the same number of effective attributes and the same declared type at each position. Attribute names and method signatures do not participate. Class-valued attribute types retain their qualified symbol identity. Copies are abstract value copies between corresponding effective positions; the backend must not copy by attribute name or alias object storage.

Known integer positions are checked statically. Unknown positions remain symbol references with their receiver and class schema for TP4. A determinable position is required when it may select compile-time storage.

Compile-time processing

Parsing retains expression fragments, complete function bodies and structured

control boundaries in dynamic RPN. Symbol references are resolved at their source occurrence, before this later pass, preserving declaration-order visibility.

‘specialize.c‘ processes these bodies with an environment per activation. It tracks known numeric values, runtime writes and pending argument copies. It does not mutate a global compile-time environment merely by parsing a function body.

Compile-time assignments update the environment and compile-time reads become constant references. Pure removed computations are discarded. If a removed assignment encloses runtime effects, those effects remain and DROP consumes the unused value. No runtime storage assignment for a compile-time symbol survives successful specialization. The final TS records main compile-time values and function-instance local compile-time bindings.

Known selections process their selected path for compile-time effects. Unknown selections retain branches and reject compile-time modifications at the responsible assignment, even if both branches would write the same value. Runtime knowledge is joined conservatively across branches. Syntax and name checks still cover the source as it is parsed, including paths later eliminated by specialization.

FROM loops with compile-time modifications are expanded while control is known. The body and its update are processed each iteration, so body changes to the control variable are respected. Other loops retain BF/BI. Calls invalidate the runtime values they may modify rather than treating unrelated state as changed. Unknown control cannot hide a compile-time modification inside a called function. Effect summaries include indexed writes and whole-object copies involving compile-time attributes. Cached calls replay only their actual effects instead of restoring unrelated captured state; known runtime writes are propagated when the body proves their values.

Source functions requiring compile-time parameters or captured compile-time state are templates in the TS. Concrete instances receive ‘$ctN‘ labels linked to their source function. Bindings and activation state remain separate between instances. An uninvoked context-dependent template does not require a concrete executable instance; its body is retained during compilation. Ordinary declared functions are emitted even when uncalled. Recursive ordinary calls reuse an in-progress instance instead of being expanded during compilation.

POUT and ordinary stores remain runtime effects. No output operation is executed by the compiler. General runtime numeric type checking is not performed by this pass. Numeric values materialized for compile-time bindings use the language’s INTEGER/SINGLEF representations, with explicit TOI when needed.

Expansion has a 100,000-iteration budget and specialization a depth limit of 128. Exceeding these reports an evaluation resource limit and suppresses output. These are compiler resource limits, not language rules requiring BY to be positive or requiring ordinary runtime recursion to terminate at compile time.

Calls and returns

FRAME prepares a pending activation. Argument expressions are evaluated in the caller and ordinary ARG destinations receive copies in the specified order. Compile-time formals are bound in the specialization environment instead. CALL selects the prepared executable instance and leaves its result as an operand. Nested calls cannot overwrite a different pending activation’s parameters. Object arguments also snapshot their compile-time attributes at the ARG copy, not after the remaining arguments have run. MATERIALIZE overlays the constants of that logical value onto its runtime fields; its OBJECT_VALUE TS record retains the original field declarations and constants. This preserves copies between compatible classes even when corresponding attributes have different compile-time qualifiers. Destination compile-time fields are bound in the compile-time environment and are not allocated or assigned by the runtime copy.

Methods retain their receiver through FRAME/CALL. Unqualified method invocations inherit the active receiver. An inherited method’s implicit attribute access and its receiver’s positional access refer to the same declaration and object state.

Return headers must name INTEGER or SINGLEF. Object-valued return expressions are rejected. This restriction does not remove class-valued parameters, object

variables, attributes or structural assignments. Numeric return compatibility continues to belong to TP4.

Verification

The current suite includes 107 original checks and 170 course scenarios, each compiled in RPN and analysis modes. Of the new scenarios, 103 are evaluated for observable output, 60 must reject the input, and seven are compile-only. Five compile-only scenarios explicitly exercise numeric checks deferred to TP4; two contain deliberately nonterminating runtime code and must still compile.

‘./run_tests.sh‘ builds with ‘-Wall -Wextra -Werror‘, requires zero Bison conflicts, and runs both verifiers. The same suites also run under ASan/UBSan with leak detection. See tp3_validation.md for references, adaptations, the defects exposed by these tests, and reproducible commands.

‘--analyze‘ runs semantic processing and specialization but suppresses RPN output. ‘--trace‘ enables debug output. Neither mode bypasses confirmed language rules. The generated RPN is still an intermediate representation; assembler generation and general runtime numeric checks remain TP4.